

LABORATORIO 04

¿Y por qué conecta y después falla?

Listeners y advertised listeners.

Guía para hacerlo por tu cuenta, paso a paso

Confluent Platform 8.2.0 · Kafka 4.2 en modo KRaft · 3 brokers

Duración estimada · 45 minutos

NovaTech Logistics · caso de estudio

Antes de empezar

Qué vas a lograr

Al final de esta guía vas a haber lanzado **dos veces el mismo cliente, en la misma red, contra el mismo broker**. Una vez va a funcionar y la otra no. **Lo único distinto va a ser la dirección por la que entra**.

Este es el error más común de Kafka en producción, y el más desconcertante, porque **la primera conexión funciona** y eso hace pensar que la configuración está bien.

LO QUE ESTE LABORATORIO DEMUESTRA

El mismo cliente, la misma red y el mismo broker. Una dirección funciona y la otra no.

Qué necesitas tener listo

Requisito	Cómo lo compruebas
Haber hecho los laboratorios 02 y 03	Hoy se da por sabido qué es el archivo <code>kafka.properties</code> y cómo se lee
Docker Desktop corriendo, con 6 GB de RAM	Ajustes de Docker Desktop, pestaña <i>Resources</i>
Git Bash abierto	<code>cd Capitulo_3/lab-04-multibroker-advertised-listeners</code>
Los puertos 9092, 9093 y 9094 libres	El paso 1 te avisa si no lo están

El caso

El equipo de **NovaTech Logistics** despliega su primera aplicación contra el clúster. En las pruebas de conexión todo responde. El equipo da el visto bueno.

En el primer despliegue de verdad, la aplicación arranca, **conecta**, y a los pocos segundos empieza a escupir errores de desconexión contra unas direcciones que nadie escribió en su configuración.

Alguien dice la frase que va a costar dos días. ***El bootstrap responde, así que Kafka está bien configurado.***

No lo está. Y el motivo es que **la dirección por la que un cliente entra la primera vez no es la que usa después**, y casi nadie sabe eso hasta que le pasa.

LA METÁFORA · LAS PUERTAS Y LA TARJETA

El **mozo** es el **broker**, el **tipo de comanda** es el **tópico**, los **sectores del salón** son las **particiones**.

El local tiene **varias puertas**. La de la calle, para los clientes. La de servicio, para los proveedores. Y una interna, para hablar entre el salón y la cocina. **Esas son los `listeners`**, y son las puertas que el local abre.

Y aparte están **las tarjetas de visita**, que dicen por dónde volver. **Esas son los `advertised.listeners`**. El local no reparte una sola tarjeta. Reparte **una distinta según a quién se la da**.

Y AQUÍ ESTÁ LA TRAMPA DEL LABORATORIO

Una de las tarjetas dice «**aquí al lado**».

Para quien está en el mismo edificio, «**aquí al lado**» es una **dirección perfectamente válida**. Para quien está en otro edificio, «**aquí al lado**» apunta a su propio edificio, **y ahí no hay ningún restaurante**. La tarjeta no está mal escrita. Está escrita **para otro público**.

En Kafka esa tarjeta dice `localhost`.

Los dos conceptos, uno al lado del otro

CONCEPTO · LISTENERS

Las **puertas que el broker abre** y en las que se pone a escuchar. Es una lista de `nombre://interfaz:puerto`. Aquí aparece el `0.0.0.0`, que significa «escucho por todas las interfaces de red que tengo».

CONCEPTO · ADVERTISED.LISTENERS

Las **direcciones que el broker le entrega al cliente** para todo lo que venga después de la primera conexión. **Nunca lleva** `0.0.0.0`, porque una dirección que se le da a otro tiene que ser una a la que ese otro pueda llegar.

LA ASIMETRÍA QUE HAY QUE RETENER

El `0.0.0.0` aparece **del lado que abre** y no aparece nunca **del lado que publica**. Si lo ves en un `advertised.listeners`, ese clúster está roto y todavía no se ha notado.

Levantar el clúster

Los comandos

```
cd Capitulo_3/lab-04-multibroker-advertised-listeners
chmod +x bin/*.sh kafka-cli/*.sh
bin/start-lab.sh
```

bin/start-lab.sh — levanta los tres brokers con los listeners ya separados. **Hoy no vas a romper nada**, porque el fallo se puede provocar sobre un clúster sano

VAS BIEN SI...

`docker ps --filter name=kafka-broker` saca **tres líneas** con `Up` , en los puertos 9092, 9093 y 9094.

SI ALGO FALLA AL LEVANTAR

Lo mismo del laboratorio 02. **Docker apagado**, ábrelo. `port is already allocated` , hay otro lab del curso arriba. **Timeout**, sube Docker a 6 GB.

Leer la tarjeta de presentación

Qué vamos a hacer

Mirar las dos líneas del broker que deciden todo lo de hoy. **Se parecen mucho y dicen cosas distintas.**

El comando

```
kafka-cli/check-listeners.sh
```

kafka-cli/check-listeners.sh — envoltorio del curso. Saca del `kafka.properties` las dos líneas que importan y te explica en qué se diferencian

EL COMANDO REAL, CON LO QUE APRENDISTE EN EL LABORATORIO 03

```
docker exec kafka-broker-1 sh -c \  
"grep -E '^(listeners|advertised.listeners)' /etc/kafka/kafka.properties"
```

LO QUE SALE PARA EL BROKER 1

```
listeners=PLAINTEXT://kafka-broker-1:29092,CONTROLLER://kafka-broker-1:39092,  
EXTERNAL://0.0.0.0:9092  
  
advertised.listeners=PLAINTEXT://kafka-broker-1:29092,  
EXTERNAL://localhost:9092
```

Cómo se leen, línea por línea

Nombre	Puerto	Para quién	Qué publica
PLAINTEXT	29092	Los otros brokers y los contenedores de la red	kafka-broker-1:29092
CONTROLLER	39092	El quórum, y solo el quórum	Nada. No aparece en la tarjeta
EXTERNAL	9092	Los clientes de tu máquina	localhost:9092

TRES COSAS QUE HAY QUE NOTAR EN ESTAS DOS LÍNEAS

Uno. El `EXTERNAL` abre en `0.0.0.0:9092` y publica `localhost:9092` . **Abre por todas las interfaces y anuncia una sola dirección.**

Dos. El `CONTROLLER` **no aparece en** `advertised.listeners` . Nadie fuera del quórum tiene por qué saber cómo llegar a él.

Tres. Los nombres `PLAINTEXT` , `CONTROLLER` y `EXTERNAL` **los eligió quien escribió el `compose`.** No son palabras reservadas de Kafka. Lo que sí es obligatorio es que cada nombre aparezca en el `listener.security.protocol.map` .

VAS BIEN SI...

Puedes decir, sin mirar, **cuál de las dos líneas lleva `0.0.0.0` y por qué la otra no puede llevarlo.**

Y ves que el `EXTERNAL` publica `localhost` . **Retén esa palabra**, porque es la que explica todo lo que viene.

El mapa · qué le devuelve el broker a un cliente

Qué vamos a hacer

Este paso no tiene comandos. Es el **modelo mental** que hace que el resto del laboratorio se **entienda**, y sin él el paso 5 parece magia negra.

CONCEPTO · *BOOTSTRAP*

La **primera conexión** que un cliente hace contra un clúster, con la dirección que tú le escribiste en su configuración. **Su único propósito es pedir el mapa**. Después de eso, esa dirección no se vuelve a usar.

Lo que pasa cuando un cliente se conecta, en tres tiempos

TIEMPO 1 · EL CLIENTE PREGUNTA

El cliente abre una conexión a la dirección de *bootstrap* que tú le diste y pide los metadatos del clúster. **En este momento el broker todavía no tiene identidad para el cliente**, y en los mensajes de error aparece como `id: -1`.

TIEMPO 2 · EL BROKER LE DEVUELVE EL MAPA

El broker contesta con **la lista de todos los brokers del clúster**, cada uno con su identificador y **con la dirección que ese broker tiene en su `advertised.listeners`**, en el listener por el que entró la consulta.

TIEMPO 3 · EL CLIENTE ABRE CONEXIONES DIRECTAS

Y aquí está todo. El cliente **descarta la dirección de bootstrap** y abre una conexión nueva **a cada dirección del mapa**. Porque cada partición tiene su líder en un broker concreto, y el cliente tiene que hablar directamente con ese.

LA CONSECUENCIA, Y ES LA DEL CASO

El bootstrap puede funcionar y todo lo demás fallar. Son dos conexiones distintas, a dos direcciones distintas, y solo la primera es la que tú escribiste.

Por eso «*el bootstrap responde*» **no dice absolutamente nada sobre si el clúster está bien configurado.** Dice que una de las dos direcciones funciona.

Mira las tres tarjetas del clúster

```
for n in 1 2 3; do
  docker exec kafka-broker-$n sh -c \
    "grep '^advertised.listeners' /etc/kafka/kafka.properties"
done
```

LO QUE SALE · EL MAPA QUE EL CLÚSTER VA A ENTREGAR

```
broker-1  advertised.listeners=PLAINTEXT://kafka-broker-1:29092,EXTERNAL://localhost:9092
broker-2  advertised.listeners=PLAINTEXT://kafka-broker-2:29093,EXTERNAL://localhost:9093
broker-3  advertised.listeners=PLAINTEXT://kafka-broker-3:29094,EXTERNAL://localhost:9094
```

VAS BIEN SI...

Entiendes que **un cliente que entre por el `EXTERNAL` va a recibir tres direcciones `localhost`** , con tres puertos distintos.

Y que uno que entre por el `PLAINTEXT` va a recibir **tres nombres de contenedor. Mismo clúster, dos mapas distintos.**

Comprobar el alcance desde tu máquina

Qué vamos a hacer

Antes de provocar el fallo, hay que establecer que **la puerta externa sí está abierta**. Si no, después no sabríamos si el problema es la tarjeta o la puerta.

El comando

```
bash -c 'exec 3<>/dev/tcp/localhost/9092 && echo "EXTERNAL alcanzable desde el host"'
```

/dev/tcp/localhost/9092 — un truco de Bash. **No es un archivo de verdad**, es una forma de abrir una conexión TCP sin instalar nada

exec 3<> — abre esa conexión en el descriptor 3. **Si el puerto no responde, el comando falla y no imprime nada**

LO QUE SALE

```
EXTERNAL alcanzable desde el host
```

VAS BIEN SI...

Salió la frase. El puerto 9092 responde desde tu máquina, que es exactamente el público para el que se escribió esa tarjeta.

Eso es lo que van a usar las aplicaciones Java del laboratorio 09, que corren en tu máquina y no en un contenedor.

SI NO IMPRIME NADA

El puerto no responde. Comprueba con `docker ps --filter name=kafka-broker` que el broker 1 esté `Up` y que en `PORTS` aparezca `0.0.0.0:9092->9092/tcp`. Si no aparece, el contenedor está corriendo pero no publicó el puerto.

El fallo clásico, en vivo

ESTE ES EL PASO DEL LABORATORIO

No vamos a romper nada. El clúster está sano y va a seguir sano. Lo único que cambia es desde dónde entra el cliente y por qué puerta.

Qué vamos a hacer

Lanzar un cliente **dentro de la red del clúster** y hacerlo entrar por el puerto **externo**, el 9092. O sea, darle a un vecino la tarjeta escrita para los de fuera.

Primero, el nombre de la red

```
kafka-cli/show-network.sh
```

kafka-cli/show-network.sh — Docker Compose le antepone el nombre del proyecto a la red, así que **no es un nombre fijo** y hay que preguntarlo

LO QUE SALE

```
novatech-lab04_mi-cluster-network
```

SI `SHOW-NETWORK.SH` NO DEVUELVE NADA

El clúster no está arriba, o su red se llama de otra forma. Sácala tú:

```
docker inspect kafka-broker-1 --format '{{range $k,$v :=  
.NetworkSettings.Networks}}{{ $k }}{{end}}'
```

Ese nombre es el que va en el `--network` del comando siguiente.

Ahora el cliente, por el puerto externo

```
kafka-cli/test-connection.sh kafka-broker-1 9092
```

kafka-cli/test-connection.sh — lanza un contenedor cliente **dentro de la red del clúster** y le hace pedir la lista de APIs del broker

kafka-broker-1 — a qué broker le habla. **Por su nombre de contenedor, que dentro de la red sí resuelve**

9092 — el puerto **EXTERNAL**. Ahí está el experimento

EL COMANDO REAL QUE CORRE POR DEBAJO

```
docker run --rm --network novatech-lab04_mi-cluster-network \
  confluentinc/cp-kafka:8.2.0 \
  kafka-broker-api-versions --bootstrap-server kafka-broker-1:9092
```

--network <red> — el cliente arranca **DENTRO** de la red del clúster, no en tu máquina. Es un vecino

--rm — el contenedor del cliente se borra solo al terminar

LO QUE SALIÓ EN LA CORRIDA MEDIDA

```
WARN [LegacyAdminClient clientId=admin-1] Connection to node 2
  (localhost/127.0.0.1:9093) could not be established. Node may not be available.
WARN [LegacyAdminClient clientId=admin-1] Connection to node 1
  (localhost/127.0.0.1:9092) could not be established. Node may not be available.
WARN [LegacyAdminClient clientId=admin-1] Connection to node 3
  (localhost/127.0.0.1:9094) could not be established. Node may not be available.

localhost:9093 (id: 2 rack: null isFenced: false) -> ERROR: DisconnectException
localhost:9094 (id: 3 rack: null isFenced: false) -> ERROR: DisconnectException
localhost:9092 (id: 1 rack: null isFenced: false) -> ERROR: DisconnectException
```

Cómo se lee, y es la lectura más importante del laboratorio

MIRA LOS IDENTIFICADORES · ID: 2 , ID: 1 , ID: 3

El bootstrap funcionó. Y se prueba con eso.

Si el bootstrap hubiera fallado, el cliente no sabría que existen tres brokers, no sabría sus números, y no sabría que ninguno está `isFenced`. **Todo eso lo aprendió del mapa**, o sea del tiempo 2 del paso 3.

Lo que falló es el tiempo 3. El cliente tomó las tres direcciones del mapa, que decían `localhost:9092` , `localhost:9093` y `localhost:9094` , y abrió conexiones contra ellas. **Y para ese contenedor, localhost es él mismo.** Ahí no hay ningún Kafka.

LO LOGRASTE SI...

Las tres líneas de error nombran `localhost` , una dirección que tú no escribiste en ninguna parte del comando.

Esa dirección se la dio el broker. **Y es correcta, para otro público.**

SI EN VEZ DE ESO VES `ID: -1`

Entonces falló **el bootstrap**, que es un fallo distinto y anterior. Ese `-1` significa «todavía no sé quién es este broker», o sea que el cliente nunca llegó a recibir el mapa.

Casi siempre es que el contenedor no arrancó dentro de la red del clúster. Comprueba el nombre de la red con `kafka-cli/show-network.sh` y vuelve a intentarlo.

SI EL COMANDO FUNCIONA Y NO FALLA NADA

Comprueba que le pasaste el puerto **9092** y no el 29092. Con el interno funciona, y ese es el paso 6.

El mismo cliente, por la puerta de dentro

Qué vamos a hacer

Cambiar un número. Nada más. El mismo comando, el mismo contenedor, la misma red, el mismo broker.

El comando

```
kafka-cli/test-connection.sh kafka-broker-1 29092
```

29092 — el puerto PLAINTEXT, el interno. Es la única diferencia con el paso 5

LO QUE SALIÓ EN LA CORRIDA MEDIDA

```
kafka-broker-3:29094 (id: 3 rack: null isFenced: false) -> (
  Produce(0): 0 to 13 [usable: 13],
  InitProducerId(22): 0 to 5 [usable: 5],
  DescribeProducers(61): 0 [usable: 0],
  ...
```

COMPARA LAS DOS SALIDAS, Y FÍJATE EN LA DIRECCIÓN

En el paso 5 el cliente recibió `localhost:9092` y no pudo llegar. **Aquí recibió `kafka-broker-3:29094`**, que es un nombre que la red de Docker sí resuelve, y llegó sin problema.

El clúster no cambió. El cliente no cambió. Cambió la tarjeta que le tocó.

LO LOGRASTE SI...

Sale **la lista larga de APIs** en vez de tres errores. **Aquí lo largo es la buena noticia.**

Y las direcciones que aparecen son **nombres de contenedor**, no `localhost`.

LO QUE ACABAS DE DEMOSTRAR

El mismo cliente, la misma red y el mismo broker. Una dirección funciona y la otra no.

La pregunta que hay que saber contestar

Qué vamos a hacer

Cerrar el laboratorio con la conversación real, la del caso.

Un colega te dice, «*el bootstrap responde, así que Kafka está bien configurado*».

Escribe tu respuesta antes de seguir leyendo. Tienes todo lo que hace falta.

LA RESPUESTA, CON LO QUE MEDISTE

El bootstrap prueba que **una** dirección funciona, que es la que tú escribiste en la configuración del cliente. **No prueba nada sobre las direcciones que el broker le va a devolver**, que son otras y no están en la configuración de nadie.

Y lo viste en pantalla. **El bootstrap del paso 5 funcionó** —el cliente supo que había tres brokers, con sus números— **y las tres conexiones siguientes fallaron**.

Y la pregunta operativa que sale de ahí

UN `ADVERTISED.LISTENERS` NO ES CORRECTO O INCORRECTO · LO ES PARA UN PÚBLICO

Público	Listener que le sirve	Qué tiene que publicar
Aplicaciones en tu máquina	<code>EXTERNAL</code>	<code>localhost:9092</code>
Otros brokers y contenedores de la red	<code>PLAINTEXT</code>	<code>kafka-broker-1:29092</code>
Clientes de otra máquina de la red corporativa	Uno más, con la IP o el nombre DNS del servidor	Ni <code>localhost</code> ni el nombre del contenedor

Esa tercera fila es la de SUNAT, y es la que no existe en este laboratorio. Un clúster que solo publica `localhost` funciona perfectamente en la máquina del desarrollador y no sirve para nadie más.

LO LOGRASTE SI...

Puedes contestar, sin mirar, **qué prueba y qué no prueba una conexión de bootstrap exitosa.**

Y sabes cuál es la primera pregunta ante un cliente que conecta y después falla. **¿Qué direcciones le está devolviendo el broker, y ese cliente puede llegar a ellas?**

Lo que aprendiste

1 La dirección por la que entras no es la que usas después.

El bootstrap sirve para pedir el mapa y se descarta. Todo lo demás va a las direcciones que el broker devolvió, que son las de su `advertised.listeners` y no están en la configuración de tu cliente.

2 «El bootstrap responde» no dice nada sobre la configuración.

Lo viste en pantalla. El bootstrap funcionó, el cliente supo que había tres brokers con sus números, y las tres conexiones siguientes fallaron contra direcciones que él no eligió.

3 Un `advertised.listeners` es correcto para un público, no en absoluto.

`localhost:9092` es perfecto para una aplicación en tu máquina e inservible para un contenedor. La pregunta nunca es «¿está bien?», sino «¿quién lo va a leer y puede llegar ahí?».

4 El `0.0.0.0` va del lado que abre, nunca del lado que publica.

En `listeners` significa «escucho por todas mis interfaces». En `advertised.listeners` no significa nada útil, porque es una dirección que otro tiene que poder usar.

Para profundizar

A · El cliente que ni siquiera llega al bootstrap

Es el fallo **anterior** al del paso 5, y conviene saber distinguirlos. Lanza el mismo cliente **sin** `--network`, o sea fuera de la red del clúster:

```
docker run --rm confluentinc/cp-kafka:8.2.0 \
  kafka-broker-api-versions --bootstrap-server localhost:9092
```

LO QUE SALIÓ EN LA CORRIDA MEDIDA

```
WARN Connection to node -1 (localhost/127.0.0.1:9092) could not be established.
WARN Bootstrap broker localhost:9092 (id: -1 rack: null isFenced: false) disconnected
java.lang.RuntimeException: Request METADATA failed on brokers
  [localhost:9092 (id: -1 rack: null isFenced: false)]
```

COMPARA ESTE ERROR CON EL DEL PASO 5, PORQUE ES EL MISMO SÍNTOMA CON OTRA CAUSA

Aquí el identificador es `-1`. El cliente nunca supo quién era ese broker, porque el bootstrap no llegó a completarse.

En el paso 5 los identificadores eran **1, 2 y 3**, o sea que el bootstrap sí funcionó.

Ese número es el diagnóstico. Con `-1`, revisa la dirección de bootstrap y la conectividad. Con un número real, revisa el `advertised.listeners`.

Y ojo con una cosa, porque este contenedor tampoco está en tu máquina. Aunque el puerto 9092 responda desde tu Mac o tu Windows, **para este contenedor `localhost` es él mismo**, igual que en el paso 5. Es la misma lección desde otro ángulo.

B · Colapsar dos listeners en el mismo puerto

Baja tu clúster de trabajo, edita `mi-cluster/docker-compose.yml` y ponle al broker 1 el mismo puerto en dos listeners:

```
KAFKA_LISTENERS: 'PLAINTEXT://kafka-broker-1:9092,CONTROLLER://kafka-broker-1:9092'
```

Levanta y mira los logs:

```
docker logs kafka-broker-1 2>&1 | tail -30
```

```
Exception in thread "main" java.lang.IllegalArgumentException: requirement
failed: Each listener must have a different port, listeners:
[PLAINTEXT://kafka-broker-1:9092, CONTROLLER://kafka-broker-1:9092]
at kafka.utils.CoreUtils$.checkDuplicateListenerPorts(CoreUtils.scala:130)
```

Lo que hay que mirar. El contenedor **termina con código 1 y no llega a arrancar**. Y el mensaje es de los buenos, porque dice exactamente qué pasa y te lista los dos listeners que chocan.

Dos listeners no pueden compartir puerto, porque el broker no tendría forma de saber qué protocolo habla cada conexión que entra. Fíjate además en dónde falla, en `checkDuplicateListenerPorts`. Es una validación que Kafka hace antes de levantar nada. **Restaura la configuración antes de seguir.**

C · Anunciar una dirección que el cliente no resuelve

Es el reto que convierte el paso 5 en un cambio de configuración de verdad. Baja el clúster y ponle al broker 1:

```
KAFKA_ADVERTISED_LISTENERS: 'PLAINTEXT://kafka-broker-1:29092,EXTERNAL://kafka-broker-1:9092'
```

Ahora el `EXTERNAL` se anuncia con el **nombre interno del contenedor** en vez de con `localhost`. Levanta, y prueba las dos direcciones:

```
kafka-cli/check-listeners.sh
kafka-cli/test-connection.sh kafka-broker-1 9092
```

Predice antes de ejecutar, que aquí está la gracia. Con esa tarjeta, ¿quién va a poder conectar y quién no? El cliente de dentro de la red, el de tu máquina, o los dos. **Escríbelo antes de correr los comandos.**

Y compruébalo con las dos pruebas que ya sabes hacer. La de dentro de la red es `test-connection.sh kafka-broker-1 9092`, y la de tu máquina es el `/dev/tcp` del paso 4 seguido de una aplicación real.

La pista está en el recorrido. En el paso 5 viste que lo que decide si un cliente llega no es el puerto, sino **si puede resolver la dirección que el broker le devolvió**. Aplica esa regla a los dos públicos y vas a saber la respuesta antes de teclear.

Y AHÍ ESTÁ LA LECCIÓN COMPLETA

Con este cambio **no arreglas nada. Cambias de público**. Por eso los clústeres de producción publican **varios** listeners a la vez, uno por cada tipo de cliente que tienen que atender. **Restaura la configuración cuando termines**.

D • El reporte del laboratorio

`plantillas/reporte-entregable.md`, con las respuestas de referencia en `soluciones/reporte-resuelto.md` y en `soluciones/respuestas-desafio.md`.

Antes de cerrar

DEJA EL TERRENO LIMPIO

Si hiciste los bloques B o C de la sección anterior, restaura primero el compose, o el laboratorio siguiente va a arrancar con un clúster mal configurado y no vas a saber por qué.

```
bin/stop-mi-cluster.sh
```

LO QUE TE LLEVAS

La próxima vez que una aplicación conecte y después falle, ya no vas a mirar la configuración del cliente. Vas a mirar **qué direcciones le está devolviendo el broker**, con `check-listeners.sh` o con un `grep` al `kafka.properties`, y vas a preguntarte si ese cliente concreto puede llegar a ellas.